

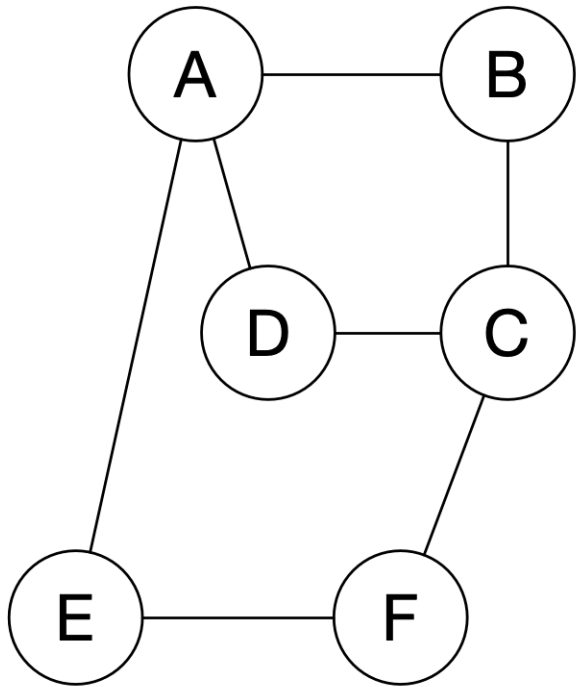
CS 3510 Algorithms: Breadth-First Search, Dijkstra's algorithm, Heaps

Aaron Hillegass
Georgia Tech

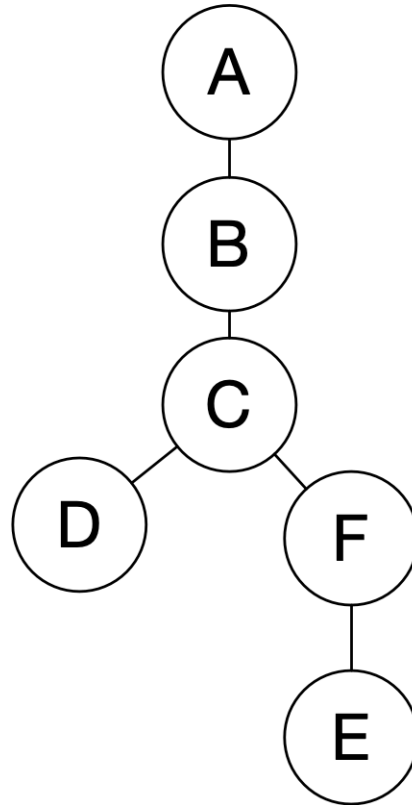
Depth-first vs Breadth-first

2/14

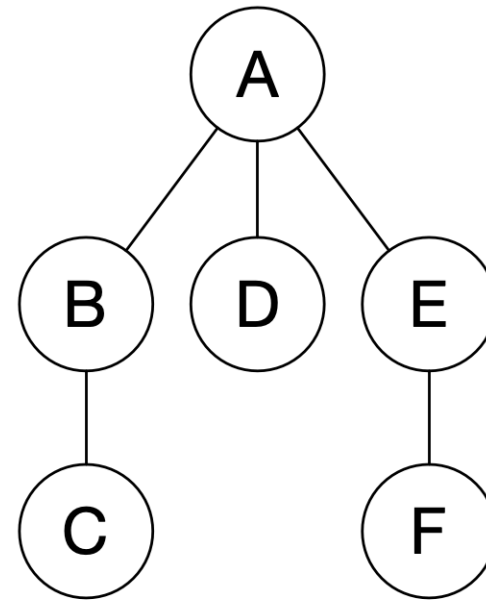
Graph



DFS



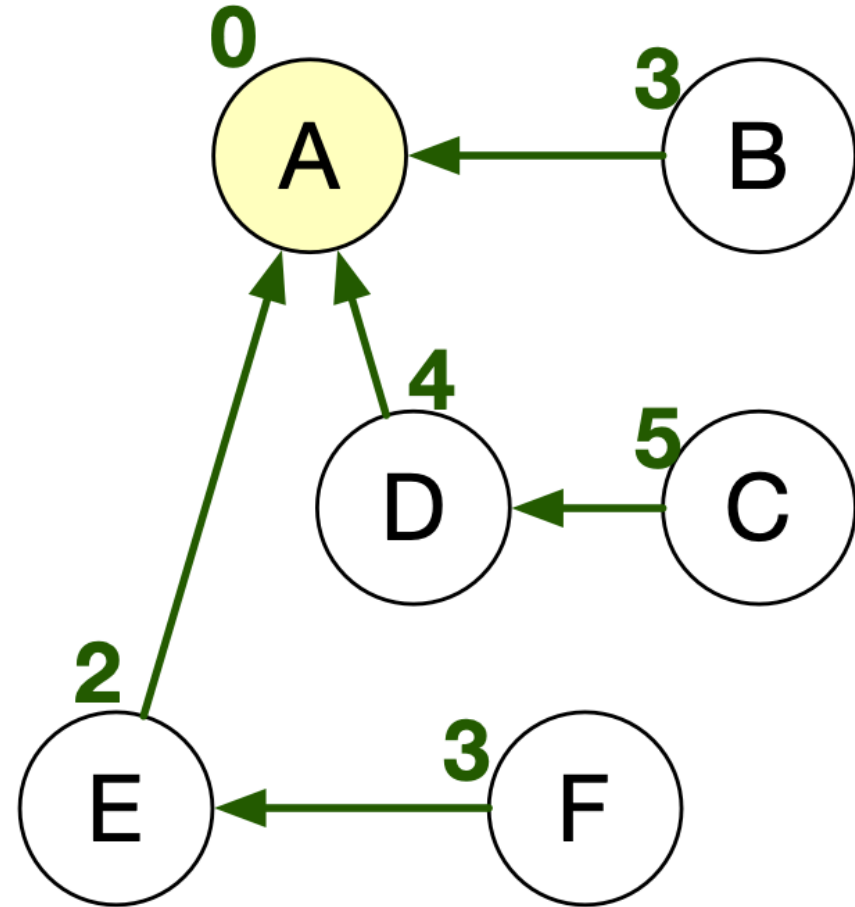
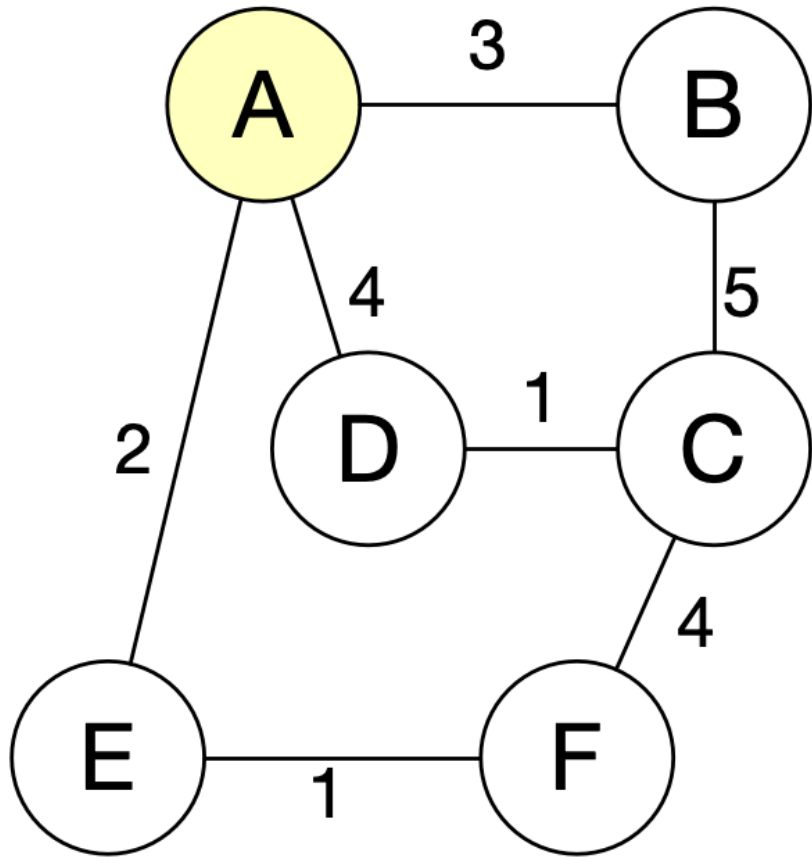
BFS



Breadth-First Search

```
1 define bfs( $G, v$ ):  
2   for all  $u \in V$ :  
3     | hops[ $u$ ] :=  $\infty$   
4   hops[ $v$ ] := 0  
5    $Q$ .push_tail( $v$ )  
6   while  $Q$  is not empty:  
7     |  $u := Q$ .pop_head()  
8     | for all edges  $(u, v) \in E$ :  
9       | if hops[ $v$ ] =  $\infty$ :  
10      | |  $Q$ .push_tail( $v$ )  
11      | | hops[ $v$ ] := hops[ $u$ ] + 1
```

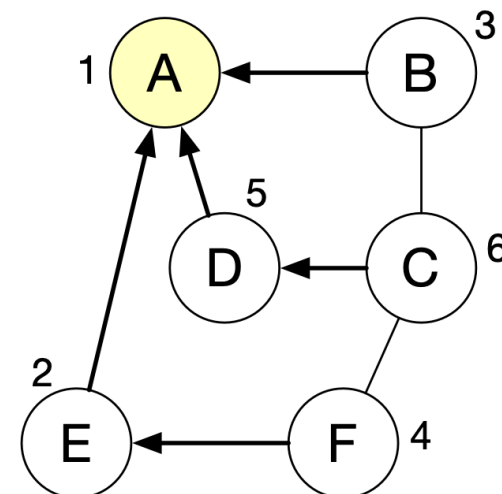
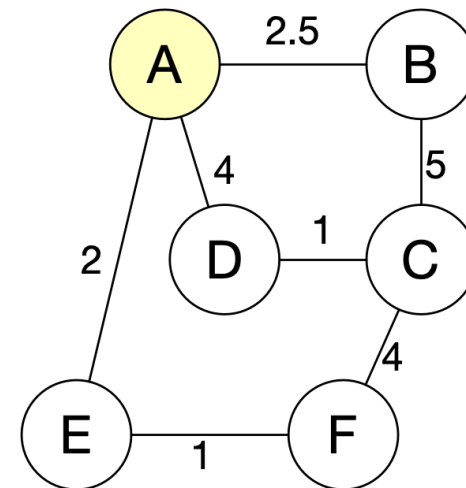
When edges have a cost

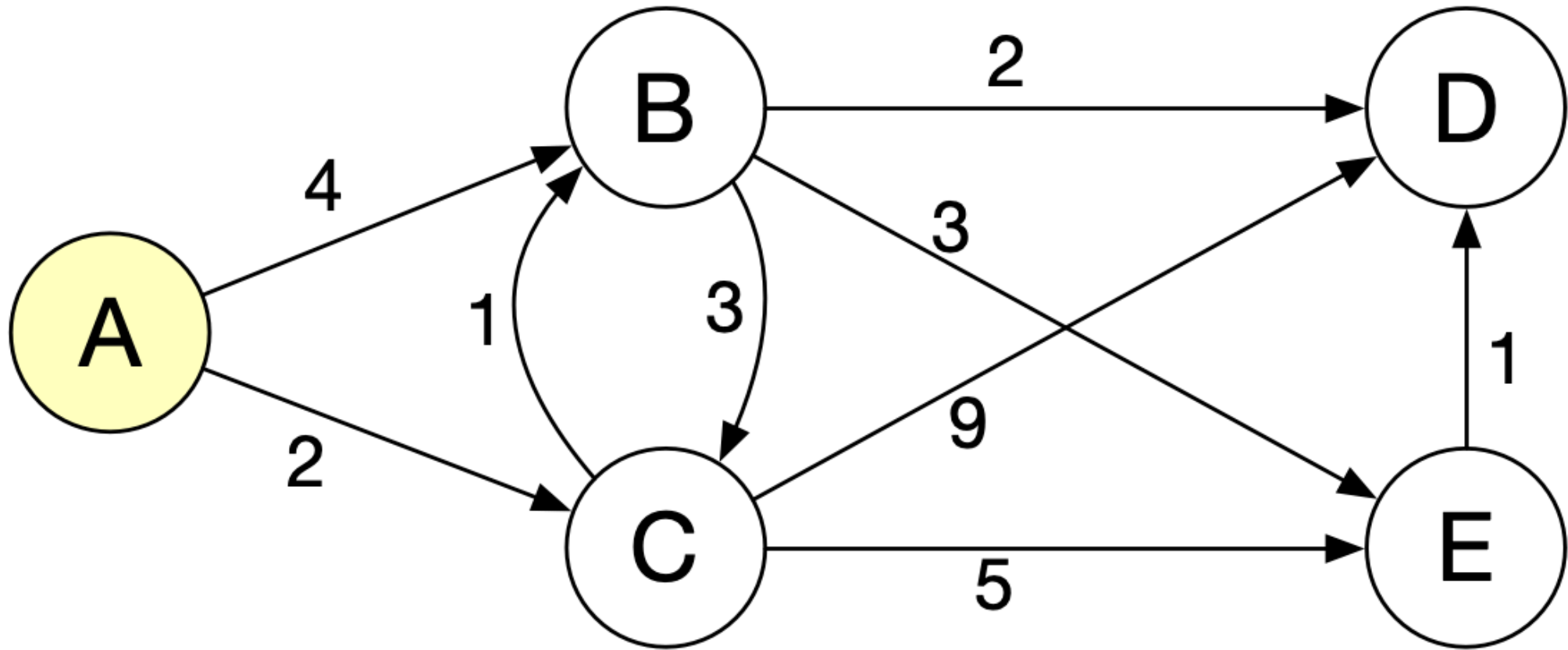


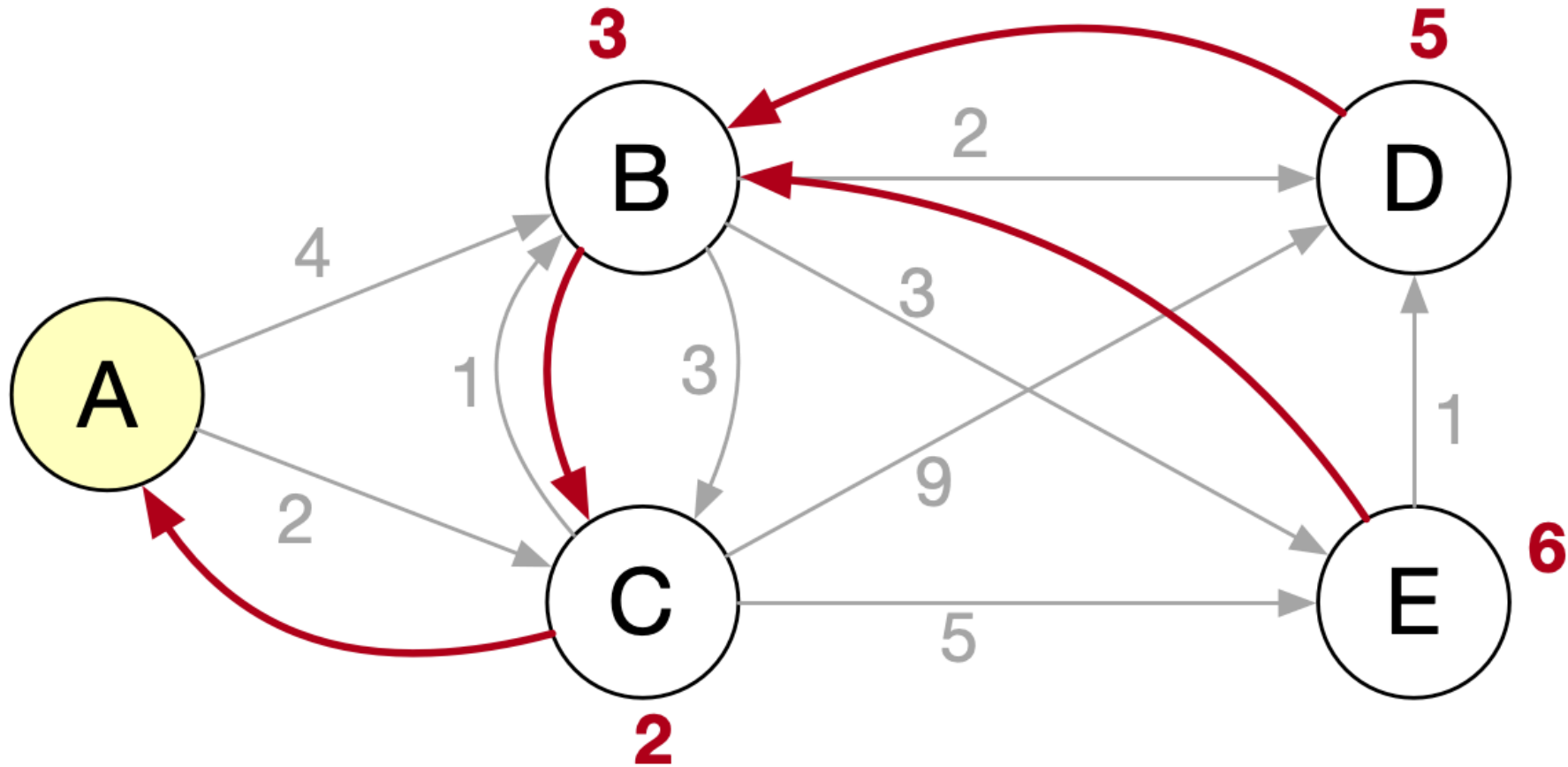
Dijkstra's Algorithm

5/14

```
1 define dijkstra( $G, v$ ):  
2   for all  $u \in V$ :  
3      $\text{dist}[u] := \infty$   
4      $\text{prev}[u] := \text{nil}$   
5    $\text{dist}[v] := 0.0$   
6    $H :=$  priority queue using  $\text{dist}$   
7   while  $H$  is not empty:  
8      $u := H.\text{pop\_min}()$   
9     for all edges  $(u, v) \in E$ :  
10       $d := \text{dist}[u] + \text{cost}[(u, v)]$   
11      if  $\text{dist}[v] > d$ :  
12         $\text{dist}[v] := d$   
13         $\text{prev}[v] := u$   
14         $H.\text{decrease}(v, d)$ 
```







Computational Complexity of Dijkstra's

```
1 define dijkstra( $G, v$ ):  
2   for all  $u \in V$ :  
3      $\text{dist}[u] := \infty$   
4      $\text{prev}[u] := \text{nil}$   
5    $\text{dist}[v] := 0.0$   
6    $H :=$  priority queue using  $\text{dist}$   
7   while  $H$  is not empty:  
8      $u := H.\text{pop\_min}()$   
9     for all edges  $(u, v) \in E$ :  
10       $d := \text{dist}[u] + \text{cost}[(u, v)]$   
11      if  $\text{dist}[v] > d$ :  
12         $\text{dist}[v] := d$   
13         $\text{prev}[v] := u$   
14         $H.\text{decrease}(v, d)$ 
```

$$|V| \times \text{pop} + (|V| + |E|) \times \text{decrease}$$

Computational Complexity of Dijkstra's

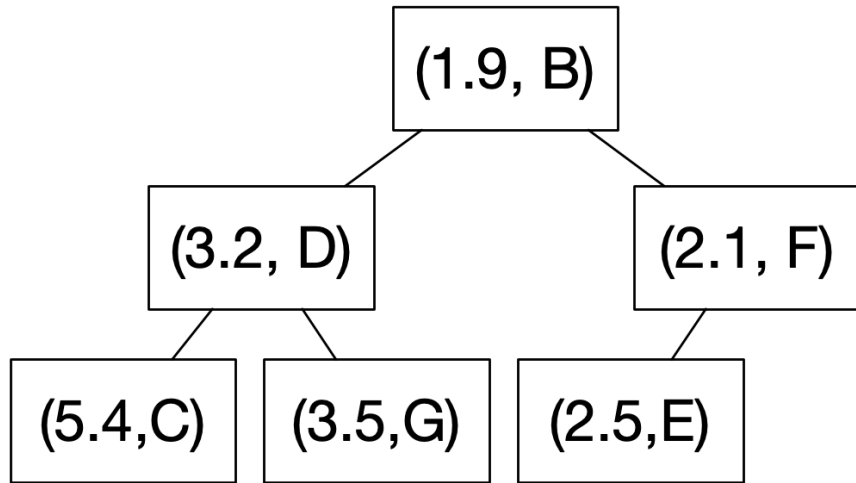
$$|V| \times \text{pop} + (|V| + |E|) \times \text{decrease}$$

Implementation	pop	decrease	Dijkstra's
Array	$O(V)$	$O(1)$	$O(V ^2)$
Binary Heap	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$
d -ary Heap	$O\left(\frac{d \log V }{\log d}\right)$	$O\left(\frac{\log V }{\log d}\right)$	$O((d V + E)\left(\frac{\log V }{\log d}d\right))$
Fibonacci Heap	$O(\log V)$	$O(1)$	$O(V \log V + E)$

Realistic? d -ary Heap where $d = \frac{|E|}{|V|}$

Binary Heap

10/14

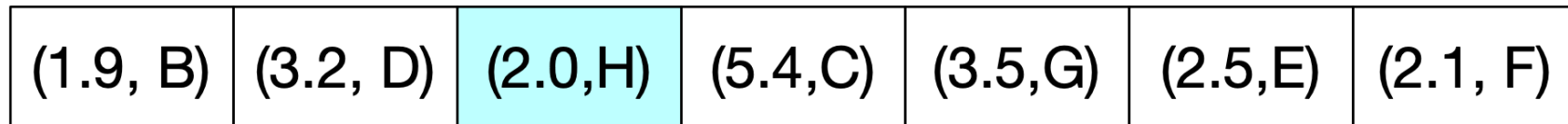
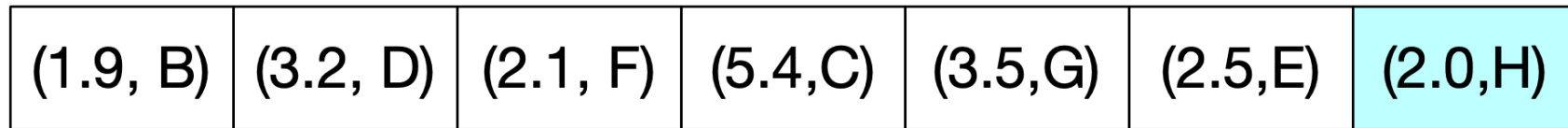
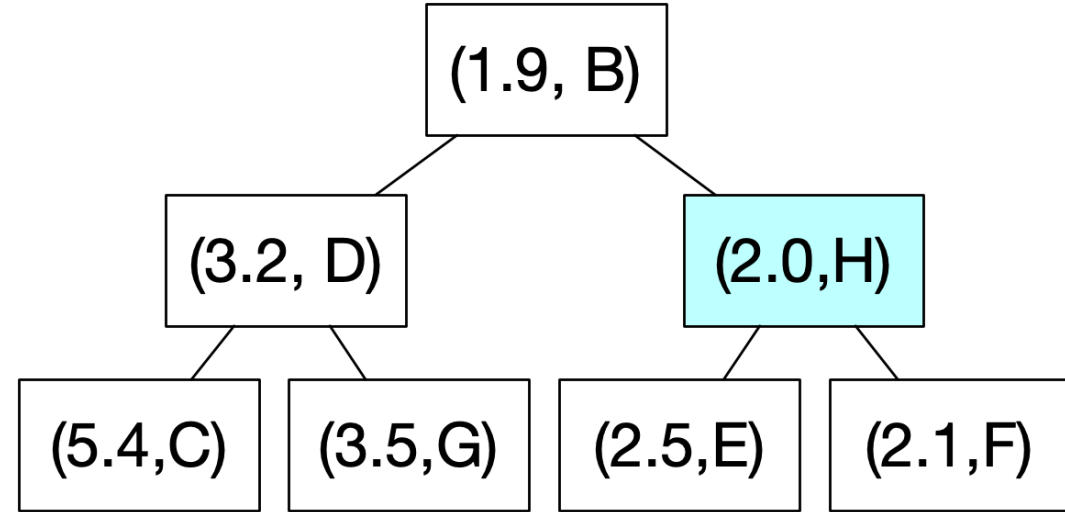
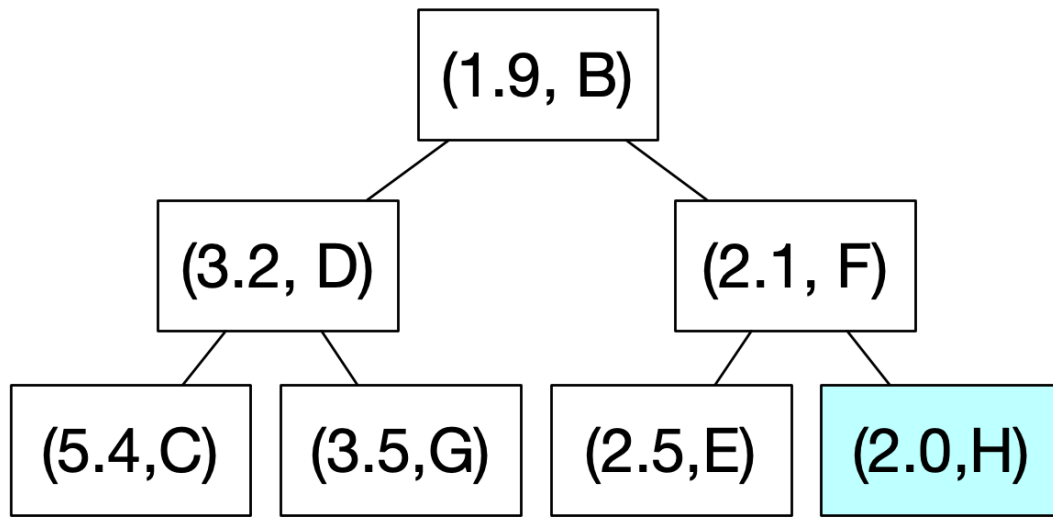


(1.9, B)	(3.2, D)	(2.1, F)	(5.4, C)	(3.5, G)	(2.5, E)
----------	----------	----------	----------	----------	----------

Parent of j ? $\left\lfloor \frac{j}{2} \right\rfloor$

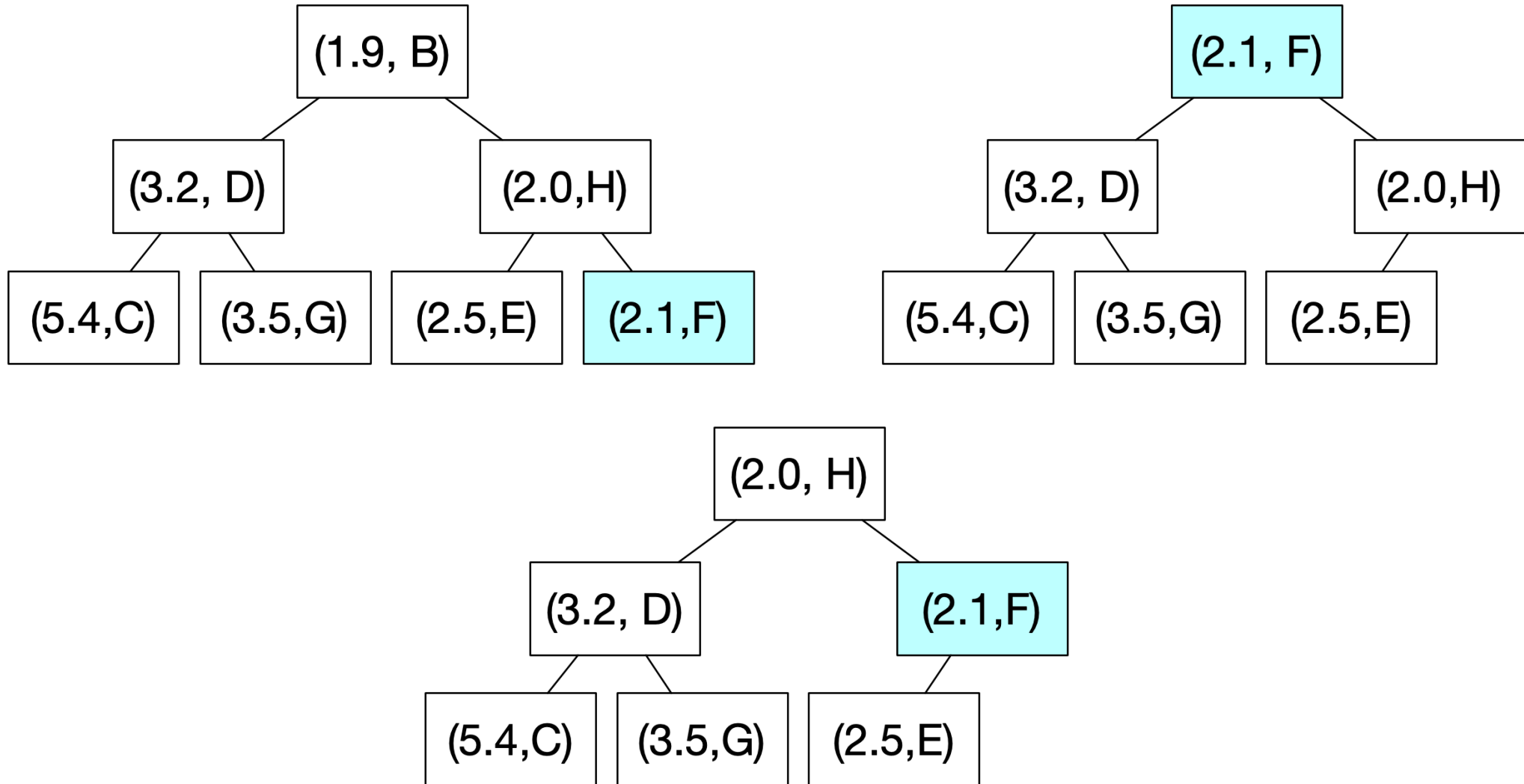
Children of j ? $2j$ and $2j + 1$

Insert into Binary Heap



Popping from Binary Heap

12/14



Each node has d children.

Parent of j ? $\left\lceil \frac{j-1}{d} \right\rceil$

Children of j ? Start at $(j-1)d + 2$

As d increases: Inserts speed up a little, deletes slow down a little.

Questions?

Slides by Aaron Hillegass